Lab Report No 5
Procedure and Stacks
Microprocessor and Interfacing techniques

Submitted By:

Huzaifa Shahbaz

21-CE-009

Submitted to:

Engr. Bushra Fiaz

Dated:

Week 05

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1:

Solution:

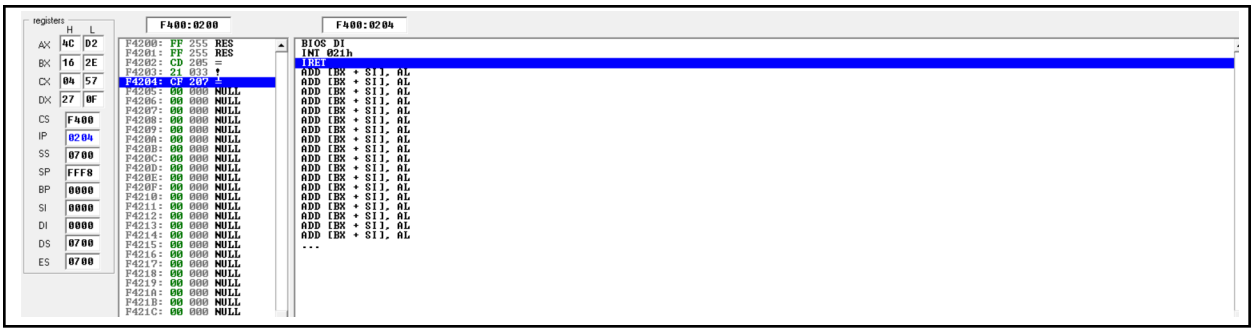
Brief description (3-5 lines)

Write a program in which all general purpose registers are utilized. During program call a procedure in which again all four general purpose registers are used. But before jumping to procedure save all registers contents by using stack operations and after returning from stack, retain the registers values. Use simple arithmetic operations.

The code

```
org 100h
section .data
section .text
start:
     mov ax, 1234
     mov bx, 5678
     mov cx, 1111
     mov dx, 9999
     push ax
     push bx
     push cx
     push dx
     call myProcedure
     pop dx
     pop cx
     pop bx
     pop ax
     mov ah, 4Ch
     int 21h
myProcedure:
     push ax
     push bx
     push cx
     push dx
     mov ax, [bp+6]
     mov bx, [bp+8]
     add cx, ax
     add cx, bx
     pop dx
     pop cx
     pop bx
     pop ax
     ret
```

The results (Screenshot)



Lab Task No 2:

Solution:

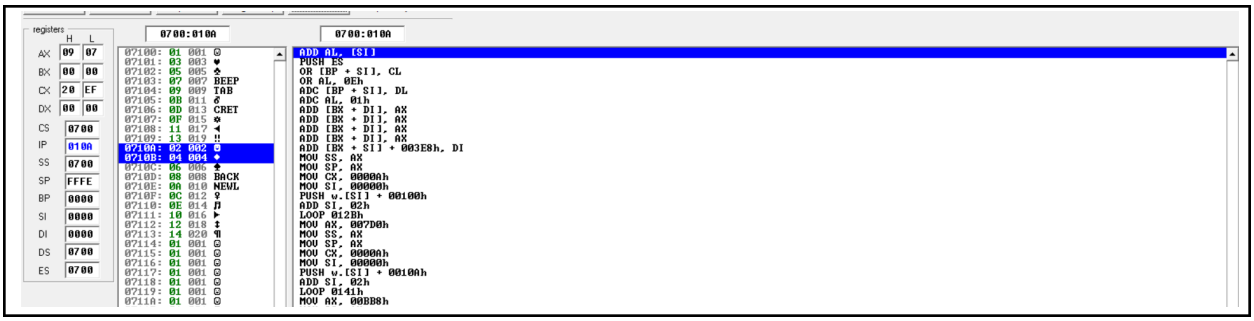
Brief description (3-5 lines)

Write a program that virtually create three different stacks. With SP= 1000; SP=2000; SP=3000 Push first 10 odd numbers to the first stack, first ten even numbers to the second stack and write al l ones to the last stack.

The code

```
ORG 100h
section .data
    odd_numbers db 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
    even_numbers db 2, 4, 6, 8, 10, 12, 14, 16, 18, 20
    ones db 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
section .text
    global _start
_start:
    mov ax, 1000
    mov ss, ax
    mov sp, ax
    mov cx, 10
    mov si, 0
push_odd_numbers:
    push word [odd_numbers + si]
    add si, 2
    loop push_odd_numbers
     mov ax, 2000
    mov ss, ax
    mov sp, ax
     mov cx, 10
    mov si, 0
push_even_numbers:
    push word [even_numbers + si]
    add si, 2
    loop push_even_numbers
   mov ax, 3000
    mov ss, ax
    mov sp, ax
    mov cx, 10
    mov si, 0
push_ones:
    push word [ones + si]
    add si, 2
    loop push_ones
    int 20h
ret
```

The results (Screenshot)



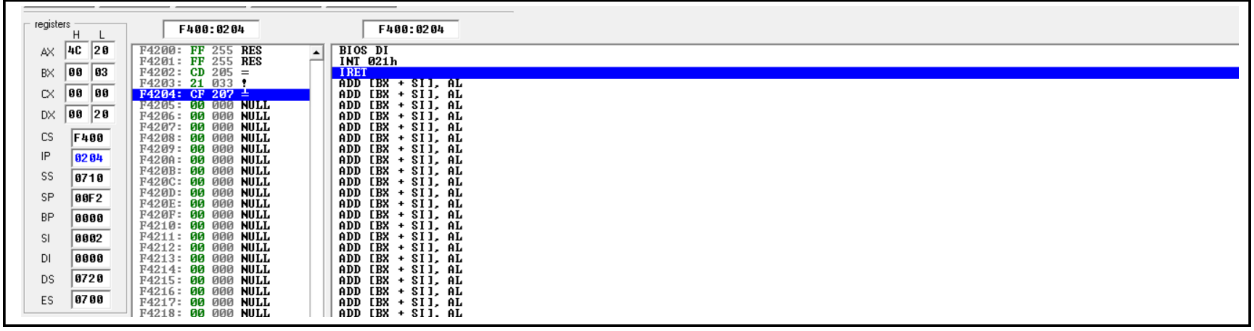
Lab Task No 3:

Solution:

Brief description (3-5 lines)
Create two stacks, swap the data of both the stacks.
The code

```
 org 100h
 .model small
.stack 100h
.data
stack1 db 10 dup(0)
stack2 db 10 dup(0)
temp db ?
.code
main proc
     mov ax, @data
     mov ds, ax
     mov ax, 1
     mov bx, 2
     push ax
     push bx
     mov ax, 3
     mov bx, 4
     push ax
     push bx
     pop ax
     pop bx
     push bx
     push ax
      mov cx, 2
     mov si, 0
     mov ah, 2
     mov dl, ' '
print_stack1:
     mov al, stack1[si]
     add al, 48
     int 21h
     inc si
     loop print_stack1
      mov cx, 2
     mov si, 0
     mov ah, 2
     mov dl, ' '
print_stack2:
     mov al, stack2[si]
     add al, 48
     int 21h
     inc si
     loop print_stack2
     mov ah, 4ch
     int 21h
main endp
end main
ret
```

The results (Screenshot)



Lab Task No 4:

Solution:

Brief description (3-5 lines)

Write a program to reverse the stored string using stack.

The code

```
org 100h
.model small
.stack 100h
.data
string db 'hello world$'
len equ $-string
stack db len dup(0)
top dw 0
.code
main proc
     mov ax, @data
     mov ds, ax
     mov cx, len
     mov si, 0
push_loop:
     mov al, string[si]
     push ax
     inc si
     loop push_loop
      mov cx, len
     mov si, 0
pop_loop:
     pop ax
     mov stack[si], al
     inc si
     loop pop_loop
      mov dx, offset stack
     mov ah, 9
     int 21h
     mov ah, 4ch
     int 21h
main endp
end main
ret
```

The results (Screenshot)